Simulation of Robotic Systems Lab 04

Lätitzia Brandt

June 12, 2025

Task 1

In the scene `Bug_P4.ttt` you see a robot with two translational degrees of freedom moving in a plane (see figure). This robot has 12 proximity sensors with a range of 30cm (you can change that in the scene file).

You should implement a motion planning algorithm based on a potential field, where you try to steer the robot in the direction to the goal (which is hard coded in the sample

For the scene `Bug_P4.ttt` you just implement the motion to goal without the influence of any obstacles. To do that you should implement the function `build_unit_direction_vector` and the function `generate_sensor_directions`

So, it returns a list of unit direction vectors for all 12 proximity sensors. Please note, that the first one is directed toward the -y direction. The following ones are placed around in mathematical positive sense (counter clockwise). You will need the ideas of this function for task 2.
The simulation is started and stopped by the python script itself.

Preparation

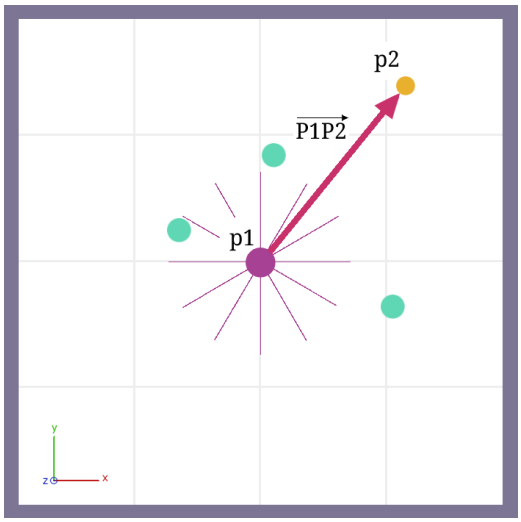


Figure 1: Unit direction vector $\overrightarrow{P1P2}$ = p2 - p1

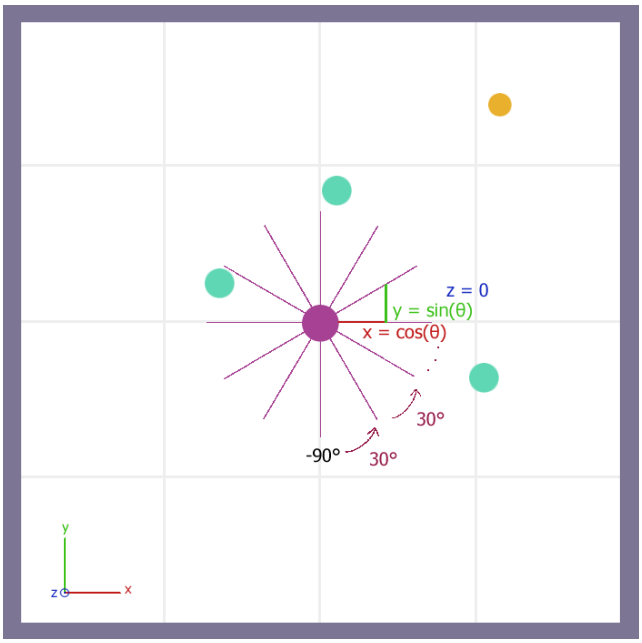


Figure 2: bug with starting_angle = -90° and 12 nbr_sensors

Implementation

Calculating unit direction vector

```
def build_unit_direction_vector(p1, p2):
    epsilon = 1e-2
    direction = np.subtract(p2, p1)

    # normalize direction vector
    norm_direction = np.linalg.norm(direction)

    # if bug is close to goal
    if norm_direction < epsilon:
        return True, np.zeros_like(direction) # returns same sized array
            with zeros

    # if not close to goal
    unit_direction = direction / norm_direction
    return False, unit_direction
```

Calculating sensor directions

```
def generate_sensor_directions(starting_angle, nbr_sensors):
    direction_vectors = []
    increase_by_angle = 360/nbr_sensors
    angle = starting_angle

    for _ in range(nbr_sensors):
        angle_rad = math.radians(angle)  # convert degrees to radians
        x = math.cos(angle_rad)
        y = math.sin(angle_rad)
        z = 0
        direction_vectors.append([x, y, z])
        angle += increase_by_angle

    return direction_vectors
```

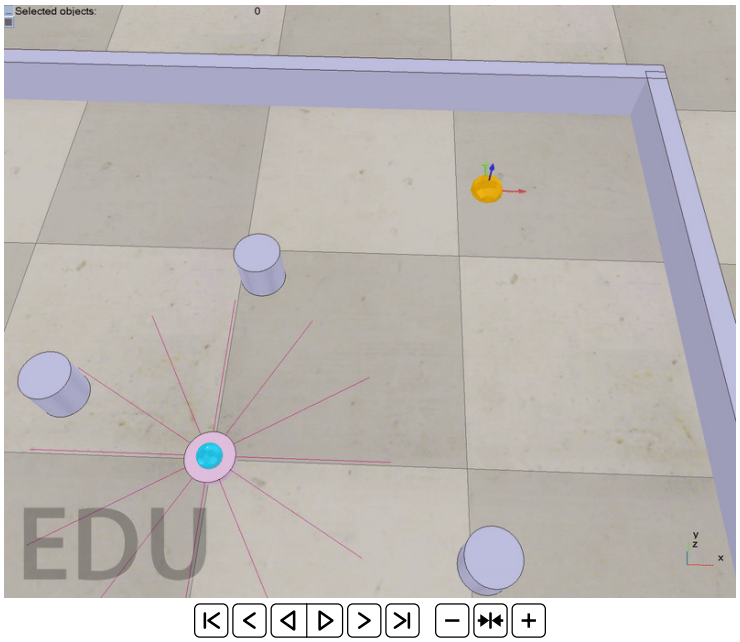
Explanation

In the first function we first calculate the direction vector through the Minkowski difference of the points p1 and p2, which are passed as parameters. Then we calculate the vector norm (positive length).
The norm is then compared to a small number epsilon. If it is smaller than epsilon, then the distance between the robot and its goal is very small. This means the robot has reached its goal. Thus we return true (goal reached) and a zero vector since the robot doesn't need to move anymore.
In the case that the robot is not yet at the goal, we calculate the unit direction vector by dividing the direction vector by its norm.

In the second function we create an empty list to hold the future direction vectors. Then we create two variables. One is the angle between the sensors (assuming they are evenly spaced in a circular arrangement), the other is the angle of the current sensor.
Inside the for-loop we determine the direction vectors by using the cosine and sine functions along with the current angle to get the x- and y-coordinates respectively. Since the robot moves in the x-y-plane the z-coordinate is set to 0. Once assembled, the vector gets added to the direction vectors list. At last the current angle gets incremented by the difference angle between the sensors. Once the loop finishes the list gets returned.

Screenshots

The bug robot moves towards the goal and reaches it.



Task 2

Now you add your script by calculating an obstacle avoidance vector in order to react to obstacles. You can move the 3 cylinders in the scene to obstruct the way to the goal.
You can even modifying the position of the cylinders, while the script is running. Try out several variations. To do that you should implement the function `generate_obstacle_avoidance_vector(detected_distances)`

The influence area of obstacles is set to 5cm, but can be variated. The list `detected_distances` contain all sensors, which are currently "firing". It is a list of a list, where the inner lists contain the index of the firing sensor as the first element and the measured distance as the second.
The simulation is started and stopped by the python script itself.

Preparation

$$F_{rep} = \begin{cases} k_{rep}(\frac{1}{d} - \frac{1}{d_0})\frac{1}{d^2}\frac{\partial d}{\partial x}, & d \leq d_0 \\ 0, & d > d_0 \end{cases}$$

Figure 3: Function for calculating the repulsive force of an obstacle

Figure 4: Example of an avoidance vector

Implementation

Function

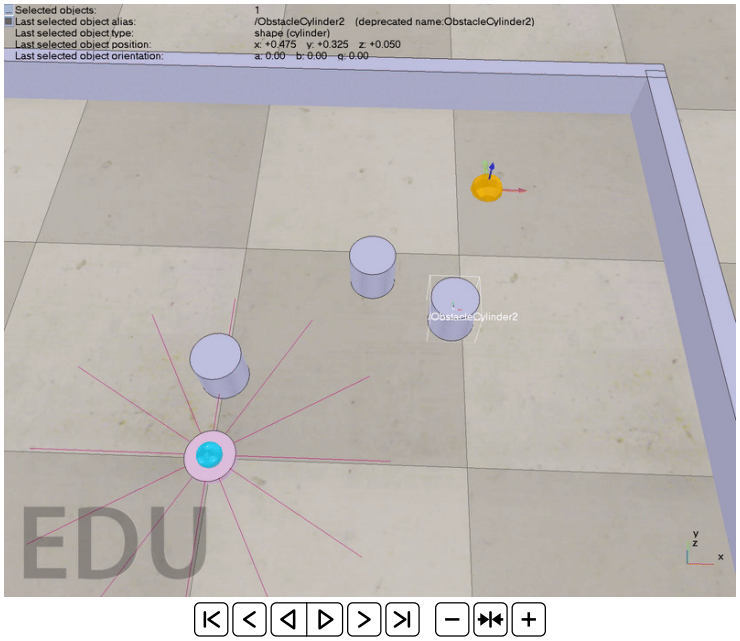
```
def generate_obstacle_avoidance_vector(detected_distances,
    direction_vectors):
    # total force of all avoidance vectors
    avoidance_vector = np.array([0., 0., 0.])
    d0 = 0.05           # 5 cm influence area of obstacles
    k_rep = 0.005       # repulsion strength constant

    for index, dist in detected_distances:
        current_distance = dist
        # if bug is too close to obstacle
        if 0 < current_distance <= d0:
            repulsive_force =
            k_rep * (1/current_distance - 1/d0) / pow(current_distance, 2)
        else:
            repulsive_force = 0.0

        sensor_direction = np.array(direction_vectors[index])
        avoidance_vector -= repulsive_force * sensor_direction
    return avoidance_vector.tolist()
```

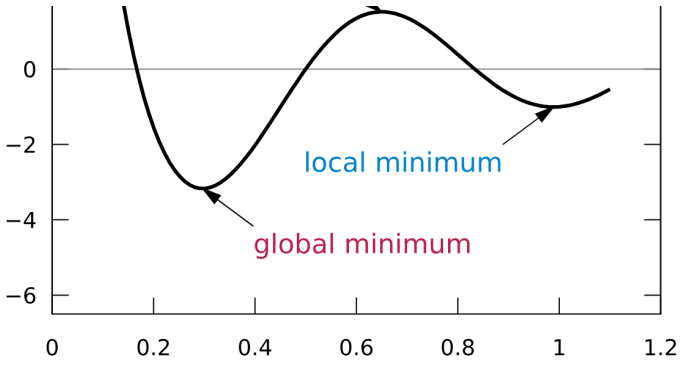
Explanation

First we define a zero array (float) to later store the difference of all avoidance vectors. In the variable d0 we set the influence area of all obstacles and in k_rep the repulsion strength of the obstacles.
Then in a for-loop we iterate over the elements in the list detected_distances which stores an index of a sensor and the detected distance dist. We check if the current distance is greater than zero and less or equal to an obstacles influence area meaning whether or not the robot is too close to an obstacle.
If this is the case we calculate the repulsive force using the function shown in figure 3.
Otherwise the force is zero.
Then we extract the direction vector of the current sensor from the directions_vector and calculate the avoidance vector by subtracting the product of the repulsive force and the sensor's direction vector.
Once the loop finishes the final avoidance vector gets returned.

Screenshots

The bug robot moves towards the goal while avoiding obstacles but gets stuck in a local minimum.





There is, however, an issue that one can run into when using the potential field approach: a so called local minimum.
The robot can get stuck in a local minimum before reaching the global minimum when the avoidance vector is the exact opposite of the goal vector, causing both vectors to cancel out and thus making the robot unable to move.
This issue is caused by the positioning of the obstacles as can be inferred from the video above.